
Printing the event's badges

The print partner interface

Our hub announces each badge on the event network;
your software collects it, prints it on your own design, and
reports back.

Spec version 1

17 September 2026

Forum Europe — event badge printing

Contents

Why this exists	3
How it works	3
1. What this interface is for	3
2. Terms	4
3. Who does what	5
4. The network	5
5. The datagram	5
6. Collecting a badge	6
7. The waiting list	11
8. Telling us how a badge went: <code>printed</code> and <code>failed</code>	12
9. Timing, cancellation and retries	13
10. Never print a badge twice	13
11. Status codes	15
12. What you may do with the data	15
13. Testing	15
14. Troubleshooting	17
15. What this interface does not offer	17
Changes	17

Why this exists

Registration and check-in for this event run on our platform, and the badges are printed on site by the print supplier. This document describes the join between the two.

Registration is designed to keep every step simple for the delegate: they register once, and everything that follows comes from that single record. At the venue, that becomes physical. A delegate gives their name at the desk and should be wearing their badge a few seconds later, with a queue forming behind them. Two things follow from that moment:

- **It has to be quick.** The desk waits 10 seconds for a badge to be collected, which is why the hub broadcasts the instant a badge is asked for, and why the printing machine belongs on a cable rather than on Wi-Fi.
- **It has to be right, once.** The badge has to pass the door scanners all week. A code that does not scan stops a delegate at a barrier, and a duplicate badge is refused at the door later on. Section 10 is the one part worth reading twice.

If anything in here would be easier in a different shape, it is usually a setting at the hub end, so it is worth asking before setup day.

How it works

A delegate checks in, and a desk iPad asks the print hub for their badge. In print partner mode the hub renders the badge, holds it, and broadcasts a small UDP message on port 8632 carrying a job id and a pickup URL. The print supplier's software fetches that URL, receives the badge as JSON, and prints it on its own printers and stock.

1. What this interface is for

It lets the event's badges be printed on a design and a printing stack that already exist. The design, the printers and the stock are all in place already; the missing piece is the delegate data. That is what each badge carries: the name, the name in the local script where one is held, the organisation, the job title, the delegate type, the QR payload the door scanners read, the badge serial and the event name.

A finished picture of the badge (`image`) comes with it, and a PDF (`pdf`) is available on request — one word and it is switched on for this event. Either one removes the need to lay anything out.

Two things must be right, whatever you print on:

- **The QR code** — encode `qr` exactly as sent, and check it scans.
- **The name** — legible at arm's length, in the script we sent it in.

Printing on your own cards

Printing onto plastic cards, on a retransfer or direct-to-card printer, is the case this interface was built for, and there is nothing extra to switch on. The printing stays exactly as it runs today; it is simply fed the delegate data.

- **Take the fields, not our picture.** `badge` (section 6) has everything: `name`, `nameLocal`, `organisation`, `job title`, `type`, `qr`, `serial`. Drop them into the card template you already have.
- **Ignore `size`, `design` and `image`.** They describe what *we* would have printed on our own label stock. They do not constrain your card, your DPI, your ribbon or your colour management.
- **Your stock is yours.** Nothing in the protocol changes with it.
- **If your cards already carry the event artwork**, print only the variable data over them. `badge.type` is the usual field to colour-code by.

If the data would be easier to handle in a different shape, it is worth asking before setup day, because it is usually a setting at the hub end.

The kit

- `print-partner-spec.md` — this document.
- `print-partner-client.py` — a working reference client: it listens, collects, saves each badge, prints (your code goes in one marked function) and reports back.
- `fake-partner-hub.py` — a stand-in hub on your own machine serving invented badges, so you can build and test without us.
- `samples/` — one real datagram and one real test badge, with its picture and the same badge as a PDF.

Python 3.10 or later, standard library only, Windows or macOS.

2. Terms

Datagram — the small UDP broadcast on port 8632 saying a badge is ready. Earlier drafts called it the announcement or the doorbell.

Badge — one delegate's badge: the JSON we serve, and the thing you print.

Job id — the hub's id for one *offer* of a badge, the `id` field. A retry of the same badge can come under a new job id.

Job key — the desk's id for the badge itself, the `jobKey` field. It survives a retry, so it is the field to dedupe on.

Collector — your software, identified by the `X-Print-Collector` it sends. One badge is served to one collector.

Collect — `GET` a badge. The first successful `GET` takes it, and the desk treats it as handed over from that moment.

Waiting list — `GET /v1/prints`: the badges nobody has collected yet.

Cancelled — a badge the hub took back because the desk gave up waiting (section 9). It answers `410 Gone` from then on, and is never printed.

3. Who does what

The hub side provides a port on the event network and a cable to it, the hub and its address on the day, one datagram for every badge a desk asks for, and the badge data itself: the name, the name in the local script, the organisation, the job title, the delegate type, the QR payload, the serial and the event. It also provides a rendered PNG, a PDF if the event has that turned on, and the stand-in hub that comes with the kit.

The printing side provides a computer on that cable, running for the whole event, which listens for the datagrams and polls the waiting list. It also provides the design, the printers, the stock, the ribbons, the drivers and the operators, along with the deduplication that keeps each badge to a single print (section 10) and a test against the stand-in hub beforehand.

Between them, that is the whole path. A question answered now is easier than one found on setup day.

4. The network

- **Use an Ethernet cable** into the event router. Wi-Fi drops broadcasts and degrades in a crowd; we give you Wi-Fi credentials as a fallback, not as the plan.
- **Everything is local.** Nothing goes over the internet, and UDP broadcasts do not cross routers — a different subnet or a guest network with client isolation will not work.
- **The hub's address** we give you on the day. It is also the source of every datagram and the host in `pickup`, and it advertises itself over Bonjour as `_fehub._tcp`.
- **Ports.** HTTP on TCP **8631**, datagrams on UDP **8632**. Your firewall needs UDP 8632 inbound and TCP 8631 outbound.
 - *Windows:* Defender asks the first time Python listens — tick **Private networks** and choose **Allow access**. If you dismissed it: Windows Security → Firewall & network protection → Allow an app through firewall, then tick Python for Private. Set the event network to **Private** as well (Settings → Network & internet → Wi-Fi → the network → Private network).
 - *macOS:* if the firewall is on, macOS asks whether Python may accept incoming connections. Choose **Allow**.
- **No authentication** — no key, no token, no password. The event network is the boundary. The routes below answer only while the hub is in print partner mode.

5. The datagram

For each badge the hub broadcasts one UTF-8 JSON datagram to UDP **8632** — to `255.255.255.255` and to the subnet broadcast address — **three times**, about 200 ms apart, on every network it is on. Expect the same badge six times or more. Every repeat carries the same `id`: act on the first, ignore the rest.

From `samples/doorbell.json`:

```
{"fehub":1,"type":"print","id":"31f3d165-c9f9-43e0-ae4e-83ce28e966c3","hub":"FE Partner Bench","pickup":"http://192.168.8.20:8631/v1/prints/31f3d165-c9f9-43e0-ae4e-83ce28e966c3","time":"2026-09-17T18:08:55.069692Z","test":true}
```

Field	Type	Meaning
fehub	number	Always 1 . Ignore a datagram without it.
type	string	Always "print" . Ignore any other value.
id	string (UUID)	The job id, identical in every repeat.
hub	string	The hub's name.
pickup	string (URL)	Where to collect the badge.
time	string	When the desk asked: UTC, ISO 8601 (2026-09-17T18:08:55.069692Z).
test	boolean	true for a test or sample badge.

The datagram carries nothing about the person. That all comes from the pickup.

Collect from the address we gave you, treating the datagram only as "there is something to fetch". A hub with two network cards can name a host in pickup you cannot reach, and a second hub on the network offers badges yours answers 404 for — those 404 s are harmless. With no address configured, use the datagram's source address with the port and path from pickup .

Datagrams get lost. Poll the waiting list (section 7) however reliable the broadcasts look.

6. Collecting a badge

```
GET http://<hub>:8631/v1/prints/<id>
X-Print-Collector: acme-print-1
```

Send your collector id on every request, and keep the same value across restarts — any text up to 100 characters. It is how the hub tells your own retry apart from a second machine that should not be printing.

The first successful GET collects the badge, and the desk counts it as handed over from that moment. **One badge goes to one collector.** Repeat the request with the same X-Print-Collector and you get the same badge back, so a dropped connection costs you nothing. A different X-Print-Collector , or none, gets 409 Conflict .

From samples/pickup.json , base64 cut short:

```
{
  "id": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3",
  "time": "2026-09-17T18:08:55.069673Z",
  "desk": "Bench page",
  "test": true,
  "badge": {
    "name": "Desk 1",
    "nameLocal": null,
```

```

    "organisation": "Sample badge – not a real event",
    "jobTitle": null,
    "type": "TEST",
    "qr": "FE1:D:CALIBRATION",
    "serial": null,
    "registrationId": null,
    "reprint": false,
    "reason": null
  },
  "event": null,
  "design": {
    "medium": "Label102x64",
    "showName": true,
    "showOrganisation": true,
    "showJobTitle": false,
    "showType": true,
    "showQr": true,
    "showLogo": false,
    "footerText": "bench sample",
    "textColor": null,
    "whiteAreaXMm": null,
    "whiteAreaYMm": null,
    "whiteAreaWidthMm": null,
    "whiteAreaHeightMm": null,
    "qrSizeMm": null,
    "layout": {
      "areaXMm": 3,
      "areaYMm": 3,
      "areaWidthMm": 96,
      "areaHeightMm": 58,
      "authored": false,
      "elements": [
        { "kind": "Name", "xMm": 2.5, "yMm": 2.5, "widthMm": 59.38, "heightMm": 9.7875, "sizeMm":
7.25, "bold": true, "align": "Left", "maxLines": 2 },
        { "kind": "Qr", "xMm": 64.38, "yMm": 14.44, "widthMm": 29.12, "heightMm": 29.12, "sizeMm":
29.12, "bold": false, "align": "Left", "maxLines": 1 }
      ]
    },
    "logo": null
  },
  "size": { "widthMm": 102, "heightMm": 64 },
  "image": {
    "contentType": "image/png",
    "dpi": 203,
    "widthPx": 816,
    "heightPx": 512,
    "base64": "iVBORw0KGgoAAAANSUhEUgAAAzAAAAIA..."
  },
  "jobKey": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3",
  "pdf": {
    "contentType": "application/pdf",
    "widthMm": 102,
    "heightMm": 64,
    "base64": "JVBERi0xLjQKJdPr6eEKMSAwIG9iago8..."
  }
}

```

(`elements` is abridged here; a real one has an entry per item.) A delegate's badge has the same shape with the fields filled: `"name": "Layla Example"`, `"nameLocal": "ليلى مثال"`, `"type": "Speaker"`, `"qr": "FE1:B:K7Q2M9"`, `"serial": "K7Q2M9"`, plus `event` and `design.logo`.

How to read the payload

UTF-8 JSON, which may contain Arabic or other non-Latin scripts — keep it UTF-8 when you save or forward it. Nullable really means nullable. Numbers may have decimals, and every length is in millimetres. Ignore unknown fields: we add fields without changing the spec version, and never remove or rename one without.

Top level

Field	Type	Meaning
<code>id</code>	string (UUID)	The job id — the same <code>id</code> as in the datagram.
<code>time</code>	string	When the desk asked: UTC, ISO 8601. May differ from the datagram's <code>time</code> by a millisecond.
<code>desk</code>	string, nullable	Which desk asked, for example <code>"Desk 3"</code> . Useful if you run a printer per desk.
<code>test</code>	boolean	<code>true</code> for a test or sample badge; nobody wears it.
<code>badge</code>	object	The delegate's data — the fields below.
<code>event</code>	object, nullable	<code>id</code> (UUID) and <code>name</code> of the event. <code>null</code> for a sample from a test hub.
<code>design</code>	object	Our own layout for this badge, for reference.
<code>size</code>	object	<code>widthMm</code> and <code>heightMm</code> of the stock the badge was drawn for.
<code>image</code>	object	The rendered badge as a PNG.
<code>pdf</code>	object, optional	The rendered badge as a PDF. Absent unless the event has asked us to send one.
<code>jobKey</code>	string (UUID)	The desk's id for this badge, stable across retries. Dedupe on this — section 10.

badge — the data you fill your design with

The part that matters if you are printing your own design. Every field is sent whether or not our design would show it.

Field	Type	What it is, and what to do with it
<code>name</code>	string, nullable	The name as registered, already cased by us — print it as sent. Usually one line; allow two, and shrink rather than truncate. Plan for 40 characters, and let 60 fit somehow.
<code>nameLocal</code>	string, nullable	The same name in their own script, usually Arabic: right-to-left , and it needs a font with Arabic coverage and shaping. Often <code>null</code> — render nothing at all rather than leaving a gap.
<code>organisation</code>	string, nullable	The longest field in practice — 60–80 characters is common. One line, shrunk or ellipsised, never wrapped over the QR code.
<code>jobTitle</code>	string, nullable	Their job title. Frequently <code>null</code> , and frequently long.
<code>type</code>	string, nullable	The delegate type: "Speaker", "Delegate", "Press", "Staff", "TEST" on a sample. The field most designs colour-code on. Per-event, so treat it as free text and have a default.
<code>qr</code>	string, nullable	The exact text to encode, and nothing else — no prefix, no URL wrapper, no trailing newline. Short ASCII (<code>FE1:B:K7Q2M9</code>), so modest error correction is fine. At least 20 mm square, with a quiet zone.
<code>serial</code>	string, nullable	The badge serial, also inside <code>qr</code> . Good for your logs. <code>null</code> on test and sample badges, so never key on it alone.
<code>registrationId</code>	string (UUID), nullable	Our id for the registration, for your logs. Not for printing. <code>null</code> on tests.
<code>reprint</code>	boolean	<code>true</code> when this badge replaces one the delegate already had.
<code>reason</code>	string, nullable	Why it is a reprint, when the desk gave a reason ("Lost badge"). Only set when <code>reprint</code> is <code>true</code> .

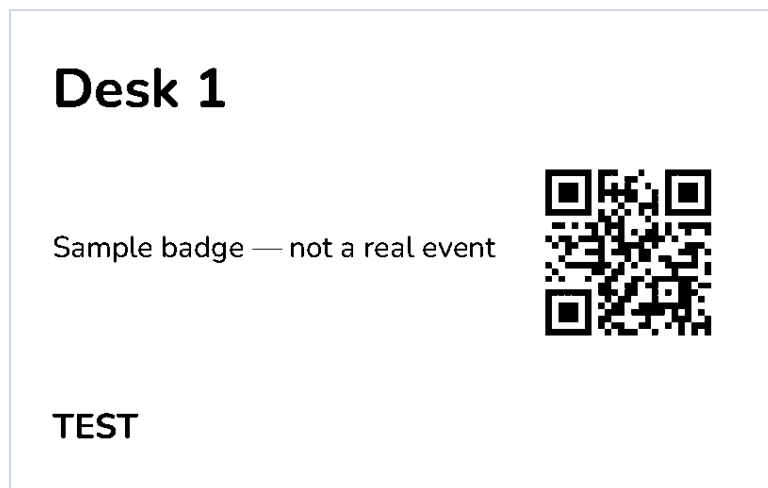
The event name is in `event.name`, and its logo, if it has one, in `design.logo`.

design — our layout, if you want to mirror it

Skip this unless you want to match our house layout. It describes how *we* would have laid the badge out on our own stock.

- `medium` — our stock: `Label102x64`, `Label89x35` or `Preprinted4x6`.
- `showName`, `showOrganisation`, `showJobTitle`, `showType`, `showQr`, `showLogo` — booleans: whether our design shows each item.
- `footerText` — fixed text along the bottom, or `null`.
- `textColor` — `#RRGGBB`; `null` means black.
- `whiteAreaXmm/Ymm/Widthmm/Heightmm` — on a pre-printed card, the blank window the text goes in, from the card's top-left. Nullable.
- `qrSizeMm` — the QR size when the event has set one.
- `logo` — the event logo: `contentType` and `base64`. Nullable.
- `layout` — `areaXmm`, `areaYmm`, `areaWidthmm`, `areaHeightmm` (the printable area from the badge's top-left), `authored` (`true` if the event's designer placed things themselves), and `elements`, one per item. Each element has `kind` (`Name`, `NameLocal`, `Organisation`, `JobTitle`, `BadgeType`, `Qr`), `xmm` / `ymm` / `widthmm` / `heightmm` measured from the top-left of the **printable area**, `sizeMm` (cap height, or the QR's side), `bold`, `align` (`Left`, `Centre`, `Right`) and `maxLines`.

image and pdf — the badge, already rendered



The finished picture as the hub sends it: pure black on white, one pixel per printer dot.

`image` is always present: the finished badge as a PNG, pure black on white, 8-bit RGB, one pixel per printer dot at `dpi` (203 unless the event uses 300 dpi kit — at 203 dpi a 102 × 64 mm badge is 816 × 512). It is what our own thermal printers burn. Print it at `dpi` with no scaling (`widthPx / dpi` inches wide) and it comes out the right size.

If you are printing cards, ignore it. It is sized for our label stock, not your card, and scaling a 203 dpi bitmap onto a 300 dpi card printer is worse than laying the fields out yourself. Use `badge` instead — or the `pdf` below, which is vector.

Field	Type	Meaning
contentType	string	Always <code>image/png</code> .
dpi	number	Dots per inch, usually 203.
widthPx, heightPx	number	Pixel dimensions.
base64	string	The PNG, base64 (standard alphabet, padded).

`pdf` is the same badge as a one-page PDF, **absent unless the event has turned it on** (a per-event setting we enable when a partner asks). Same layout as the PNG, but vector, with the fonts embedded (Arabic included), and the page is the stock's size, so there is no DPI to guess. Take `widthMm` and `heightMm` as the real dimensions — the page box is those rounded to whole points, so it can sit a fraction of a point inside them. Print at 100%, no scaling, no fit-to-page.

Field	Type	Meaning
contentType	string	Always <code>application/pdf</code> .
widthMm, heightMm	number	The page size, matching <code>size</code> .
base64	string	The PDF, base64.

The PDF is roughly five times the PNG, a few hundred kilobytes — once, at collect. The waiting list stays small either way: it lists ids and URLs, never payloads.

7. The waiting list

```
GET http://<hub>:8631/v1/prints
```

```
{
  "waiting": [
    {
      "id": "24342316-0fe8-46f9-b509-9c83aaf88cf6",
      "jobKey": "24342316-0fe8-46f9-b509-9c83aaf88cf6",
      "time": "2026-09-17T10:37:09.690477Z",
      "test": true,
      "pickup": "http://192.168.8.20:8631/v1/prints/24342316-0fe8-46f9-b509-9c83aaf88cf6"
    }
  ]
}
```

Every uncollected badge, oldest first. The `pickup` URL is built from the host you called, so it is always one you can reach.

Poll it every 5 seconds and collect anything you do not already have — your safety net for a lost datagram and for the first seconds after start-up. The desk waits only 10 seconds (section 9), so a longer interval loses badges. Entries carry `jobKey`, so you can skip a badge you have already printed **without collecting it at all**.

8. Telling us how a badge went: printed and failed

```
POST http://<hub>:8631/v1/prints/<id>/printed
POST http://<hub>:8631/v1/prints/<id>/failed
X-Print-Collector: acme-print-1
Content-Type: application/json

{"reason": "out of ribbon"}
```

Call one, once per badge, after your printing has returned or failed. The body is optional and only `failed` uses it: `reason` is free text, up to 200 characters, shown to our desk staff word for word. If you cannot send the header, `{"collector": "acme-print-1"}` in the body works instead.

Nothing depends on them. A client that never calls either route behaves exactly like one that does, and no badge waits for a report. The reference client sends them by default; take that out and nothing else changes.

What we do with them. A `failed` report reaches the desk straight away, shown like any other print failure with your reason in the sentence — "Desk 1: the print partner reported a failure — out of ribbon" — and we offer the operator a reprint, which comes back to you as a new badge with a new `jobKey`. Without it, a jam and a good print look identical from the desk: the iPad says printed, and the steward turns round to hand over something that never came out of the printer. A `printed` report closes the badge quietly.

The rules, if you use them.

- Report on a badge **you** collected. Anyone else gets `409`, and so does a report on a badge nobody collected.
- **The first report wins, and repeats are safe** — a repeat gets `200 OK` with `"applied": false`, `"state": "already reported"`. A later, contradicting report changes nothing. Report once, when you know.
- **A late report is accepted.** If the desk had given up (section 9) you get `200 OK`, `"applied": false`, `"state": "cancelled"`. Not an error, and no retry.
- An `id` the hub never held or has forgotten answers `404`. The record lives 10 minutes from collection, which is the window a report has to land in.
- A report never re-sends a badge. It puts the failure in front of a human.

The reply:

```
{"id": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3", "outcome": "failed", "recorded": true, "applied": true, "state": "recorded", "reportedOutcome": "failed"}
```

Field	Type	Meaning
<code>id</code>	string (UUID)	The badge you reported on.
<code>outcome</code>	string	"printed" or "failed" — the route you called.
<code>recorded</code>	boolean	Always <code>true</code> on a <code>200</code> : the report is on the badge's record.
<code>applied</code>	boolean	<code>true</code> when the report changed something. <code>false</code> for a repeat or a badge the desk had already given up on.
<code>state</code>	string	"recorded", "already reported" or "cancelled".
<code>reportedOutcome</code>	string, nullable	The outcome now standing on that badge, which for a repeat is the <i>first</i> one you sent.

If a report call itself fails, treat it like a failed log write: note it and carry on. Never retry in a loop, and never hold up the next badge for it.

9. Timing, cancellation and retries

- **The desk waits 10 seconds.** In time, and it shows as printed; otherwise the desk shows "Not picked up by the print partner."
- **The hub then cancels the badge** and it answers `410 Gone` from then on. Do not print it and do not ask again — the desk may already have printed a replacement.
- A collect and a cancellation cannot both happen: your `GET` either lands first or gets `410`.
- **A badge is deleted 10 minutes** after collection, or 10 minutes after it was made if nobody collected it. It answers `404` after that.
- **A hub restart cancels every uncollected badge.**

When a desk asks for the same badge again, one of three things happens: you already collected it and nothing is announced; the hub had cancelled it, so it is re-announced under a **new `id`** with the **same `jobKey`** — collect and print that one; or it is still waiting and the same `id` is announced again. One desk request never leaves you holding two live job ids.

Three cases that *are* new badges and must each be printed: a **reprint** (`"reprint": true`, new `jobKey`, new `id`, usually a new `serial` and a `reason` — somebody is at the desk with nothing); a **badge per day** at events that issue one; and a **test badge** (`"test": true`), which you can print or just collect.

10. Never print a badge twice

A delegate holding two badges is the complaint we hear at the desk, and the older one is what stops them at the door. Repeats are not a fault: we announce each badge three times on every network and you poll every 5 seconds, so one badge reaches you five or six times on an ordinary morning. Printing it exactly once is as much your job as ours.

What we guarantee

- **One badge, one id**. Every datagram and every waiting-list entry for a badge carries the same `id`, and the hub never reuses an `id` for a different badge.
- **One collect**. The first `GET` takes the badge; a different collector gets `409` rather than the badge, and your own repeat `GET` gets the same bytes, so a lost reply never costs you a badge.
- **One live badge per desk request**. A desk that asks twice because it heard nothing back does not create two badges.
- **jobKey survives a retry**. A badge re-offered after a cancellation has a new `id` and the same `jobKey`; the old `id` is already `410`.
- **410 is final** for that `id`.

What you must do

1. **Dedupe on id** — ignore a datagram for an `id` you already hold or have printed.
2. **Dedupe on jobKey** too. One `jobKey` is one badge however many `ids` it appears under, and because the waiting list carries `jobKey` you can skip a badge you have printed without collecting it.
3. **Keep that record on disk and read it at start-up**, so a restart or a crash does not reprint this morning's badges. The reference client keeps `printed.json` beside the badge folder.
4. **Write the record before you send anything to a printer, not after**. If your software dies in between, the badge goes unprinted — the failure you want, because a human can see it, whereas a duplicate walks out of the building.
5. **Send X-Print-Collector**, the same value every time.
6. **Run one collector**. One computer, one process, one queue. A spare machine left running, or yesterday's process nobody stopped, is the usual explanation for everything printing twice. A spare needs its own `X-Print-Collector` so the hub refuses it with `409` — a safety net, not a design.

The fields to key on

- **id** — in the datagram, the waiting list and the badge. It identifies this *offer* of a badge. Ignore repeats of it.
- **jobKey** — in the waiting list and the badge. It identifies the *badge*. One `jobKey` is one printed badge, permanently.
- **badge.serial** — in the badge. The serial that is also inside the QR code: good for your logs and for answering questions after the event. It is `null` on test and sample badges, so never key on it alone.

If your printer jams after you collected the badge, the desk has already moved on and there is no way to hand it back. Clear the jam and print from the copy you saved — that is why the reference client writes the files before printing. If you cannot, the desk must print a replacement: a `failed` report (section 8) tells them, and their reprint arrives as a new badge with a new `jobKey`. **Never retry by collecting again.**

Test this on purpose. `fake-partner-hub.py` tries to make your client print twice. It announces everything three times, re-offers one badge under a new `id` with the same `jobKey` after you collect it ("Robin Retry"), and leaves another on the waiting list for ever after collection ("Ash Repeat"). Run your one collector, restart it mid-run, and check each badge came out exactly once. (Two copies with separate records can each print a re-offered badge — which is why you run one.)

11. Status codes

Errors are `application/problem+json` with a plain-English `detail`:

```
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.11","title":"Print cancelled","status":410,"detail":"This print was cancelled because it wasn't collected in time."}
```

Status	What it means	What to do
200	The badge (collect), the list (waiting list), or your report was taken.	Print it; collect what you do not have; carry on.
404	No badge with that id on this hub — it was never here, or its 10 minutes are up.	Drop it. On a report, nothing more to do.
409	Two cases, told apart by <code>title</code> . " Not in print partner mode ": the hub is driving its own printers. " Already collected " / " Not your badge ": another collector holds that badge.	Not in partner mode: keep polling every 5 s and tell us if it persists. Another collector: do not print it , and find the other collector (section 10).
410	The hub cancelled the badge because nobody collected it in time.	Do not print it. Drop it and never ask again.
503	The hub is locked because nobody has started the desk yet today.	Keep polling every 5 s.

If you cannot reach the hub at all, check you are on the event network, then retry with a growing delay. The reference client starts at 5 s and doubles to a maximum of 60 s.

12. What you may do with the data

- You receive only what goes on a badge, and only for badges a desk has asked for. There is no attendee list, no search and no check-in data.
- Keep it only as long as you need it to print. Delete the saved badges — the reference client's PNG, PDF and JSON files — at the end of the event.
- Do not send it anywhere else, and do not use it for anything but printing.

13. Testing

On your own machine. In one window, `python fake-partner-hub.py` (Windows: `py -3 fake-partner-hub.py`); in another, `python print-partner-client.py --hub 127.0.0.1`.

The client saves three waiting badges straight away, finds a fourth only through the waiting list, and refuses a fifth that has been cancelled — all correct. For more, open `http://127.0.0.1:8631/` and use **Send a sample print**; that page shows whether you collected within 10 seconds and what you reported. The stand-in hub also takes `--every 20`, `--mode manage (409)` and `--mode locked (503)`.

Worth doing deliberately: stop your client, send a sample badge, wait 10 seconds, start the client again, and confirm that badge does **not** print.

The PDF path. The stand-in hub sends a `pdf` on every badge: collect one and print the saved `<id>.pdf` on the stock you intend to use. `--no-pdf` behaves like an event with the setting off.

`samples/pickup.pdf` is a badge our own hub produced, if you would rather print one before you write anything.

There is no joint test before the event. The two systems meet for the first time at the venue on setup day, and that is exactly why the stand-in hub is in the kit. It sends the same datagrams and serves the same badges as the real one, on the same ports and with the same status codes, so a system that works against it will work against ours. If something does not behave, tell us and we will look at it.

Have ready on setup day: the machine that will run all event, already working against the stand-in hub; your printers, stock and consumables; an Ethernet cable for our router; and somebody who can change your settings on the spot.

On the day, the hub is put into partner mode and sends a test badge. It also serves a page at `http://<hub>:8631/` offering **Use print partner mode** and **Send a sample print**, along with the time of the last collection and the number of badges waiting. That page can sit open on a screen by the printers during set-up.

Our team is on site throughout. If anything looks wrong, please come and find us rather than working around it.

14. Troubleshooting

- **Nothing arrives, and you cannot reach the waiting list.** You are not on the event network, the hub address is wrong, or something is blocking TCP 8631.
- **The waiting list works, but no datagrams arrive.** A firewall is blocking UDP 8632 inbound — on Windows, the network is set to Public, or Python is not allowed through — or you are on Wi-Fi that isolates clients. Use the Ethernet cable. Badges still arrive through the waiting list, just later.
- **"cannot listen on UDP 8632".** Another program holds the port exclusively. Close it.
- **Constant 409, "Not in print partner mode".** The hub is not in partner mode. Ask us to switch it.
- **409, "Already collected" or "Not your badge".** Another collector has that badge: a spare machine, an old process, or a second copy of your software. Stop the extra one, and do not print the badge.
- **A delegate has two badges.** Two collectors, or a client with no durable record of what it printed. Read section 10.
- **503.** Nobody has started the desk yet. It clears when they do.
- **The desk says "Not picked up by the print partner".** You did not collect within 10 seconds. Check your software is running and on the network.
- **410.** The hub cancelled the badge. Refusing to print it is the correct behaviour.
- **An Arabic name looks wrong in a saved file.** Save it as UTF-8. Your console may render it badly even when the file itself is correct.

15. What this interface does not offer

- No attendee list, no search, no check-in data.
- No printer telemetry: queue depth, ink levels and the state of your kit are yours. The section 8 routes are per badge and optional; nothing else points back at us.
- No other part of the hub, and nothing over the internet.

Changes

- **1 (17 Sep 2026):** the first version. It covers the datagram, the pickup, the waiting list, every field and status code, the optional `pdf` alongside `image`, the two optional routes for telling us how a badge went (`printed` and `failed`), the rule that one badge is printed once, and how to test against the stand-in hub before setup day.